

JEGYZŐKÖNYV

Web technológiák II.

Féléves feladat

Terméknyilvántartó rendszer

Név: **Kovács Attila Marcell**

Neptunkód: **SEGUV3**

Dátum: **2026. 05. 31.**

Tartalomjegyzék

1. Bevezetés	4
2. Motiváció	4
3. Alkalmazott webes paradigmák.....	5
3.1 Kliens–szerver architektúra.....	5
3.2 Single Page Application (SPA).....	5
3.3 REST API	5
3.4 Komponens alapú fejlesztés.....	5
3.5 CRUD műveletek.....	5
4. Felhasznált technológiák	6
4.1 Angular	6
4.2 Angular Material	6
4.3 Node.js	7
4.4 Express.js	7
4.5 MongoDB	7
4.6 Mongoose.....	8
5. A rendszer felépítése.....	8
5.1 Frontend	8
5.2 Backend.....	9
5.3 Adatbázis.....	9
5.3.1 Users.....	9
5.3.2 Products	9
5.4 Projektstruktúra.....	10
5.4.1 Backend.....	10
5.4.2 Frontend	10
6. Adatbázis tervezése	11
6.1 Users kollekció.....	11
6.2 Products kollekció.....	12

6.3 Adatkapcsolatok.....	13
7. Megvalósított funkciók.....	13
7.1 Regisztráció.....	13
7.2 Bejelentkezés	14
7.3 Navigációs fejléc és felhasználói információk	14
7.4 Útvonalvédelem (Auth Guard).....	15
7.5 Termékek megjelenítése	15
7.6 Új termék hozzáadása	16
7.7 Termék módosítása	16
7.8 Termék törlése	16
7.9 SnackBar értesítések.....	17
7.10 Statisztikai kártyák.....	17
7.11 Kategória szerinti szűrés	18
8. Fontosabb kódrészletek bemutatása	19
8.1 Felhasználó regisztrációja	19
8.2 Felhasználó bejelentkezése	20
8.3 Termék betöltése.....	20
8.4 Termék hozzáadása / módosítása.....	21
8.5 Kategória szerinti szűrés	21
8.6 Kategóriák színezése.....	22
8.7 Statisztikai kártyák számítása	23
8.8 Kijelentkezés.....	23

1. Bevezetés

A webes alkalmazások napjainkban egyre nagyobb szerepet töltenek be az adatok kezelésében és nyilvántartásában. A különböző vállalatok és szervezetek számára fontos, hogy a szükséges információk gyorsan és hatékonyan elérhetők legyenek, valamint az adatok kezelése egyszerű és biztonságos módon történjen.

A féléves feladat keretében egy terméknnyilvántartó rendszer került megvalósításra Angular, Node.js és MongoDB technológiák felhasználásával. Az alkalmazás célja, hogy a felhasználók egy modern webes felületen keresztül kezelhessék a nyilvántartott termékeket, valamint azok legfontosabb adatait.

A rendszer lehetőséget biztosít felhasználói regisztrációra és bejelentkezésre, továbbá a termékek adatainak létrehozására, megjelenítésére, módosítására és törlésére. A fejlesztés során kiemelt szempont volt a könnyen kezelhető felhasználói felület kialakítása, az adatok biztonságos tárolása, valamint a félév során tanult technológiák gyakorlati alkalmazása.

A projekt kliens-szerver architektúrát alkalmaz, ahol az Angular alapú frontend felel a felhasználói felület megjelenítéséért, míg a Node.js backend biztosítja az üzleti logikát és az adatbázissal való kommunikációt. Az adatok tárolása MongoDB adatbázisban történik.

2. Motiváció

A féléves feladat elkészítéséhez egy terméknnyilvántartó rendszer megvalósítását választottam. A téma azért bizonyult megfelelőnek, mert egy ilyen alkalmazás jól szemlélteti a webalkalmazások fejlesztése során alkalmazott legfontosabb funkciókat, például a felhasználói regisztrációt, a bejelentkezést, az adatbázis-kezelést, valamint az adatok létrehozását, módosítását, törlését és megjelenítését.

A projekt elkészítése lehetőséget biztosított arra, hogy a félév során tanult technológiákat gyakorlati környezetben alkalmazzam. Célom egy olyan modern megjelenésű és könnyen kezelhető rendszer létrehozása volt, amely megfelel a feladatkiírás követelményeinek, ugyanakkor valós webalkalmazások működését is szemlélteti.

A fejlesztés során külön figyelmet fordítottam a felhasználói élményre, az áttekinthető kezelőfelület kialakítására, valamint a kliensoldali és szerveroldali működés elkülönítésére. Fontos szempont volt továbbá, hogy az alkalmazás ne csak alapfunkciókat tartalmazzon, hanem kiegészüljön olyan elemekkel is, mint a regisztráció, az útvonalvédelem, a visszajelző üzenetek, a statisztikai kártyák és a kategória szerinti szűrés.

3. Alkalmazott webes paradigmák

3.1 Kliens–szerver architektúra

A fejlesztett alkalmazás kliens–szerver architektúrát alkalmaz. A kliensoldalt az Angular alapú frontend valósítja meg, amely a felhasználói felület megjelenítéséért felel, míg a szerveroldalt a Node.js és Express.js alapú backend biztosítja. A két komponens HTTP kéréseken keresztül kommunikál egymással.

3.2 Single Page Application (SPA)

Az alkalmazás Single Page Application (SPA) elven működik. Ennek lényege, hogy az oldal tartalma újratöltés nélkül frissül, így a felhasználó gyorsabb és folyamatosabb használati élményt kap. Az oldalak közötti navigációt az Angular Router valósítja meg.

3.3 REST API

A frontend és a backend közötti kommunikáció REST API segítségével történik. A backend különböző végpontokat biztosít a felhasználók hitelesítésére, valamint a termékek létrehozására, lekérdezésére, módosítására és törlésére. Az adatok JSON formátumban kerülnek továbbításra.

3.4 Komponens alapú fejlesztés

Az Angular keretrendszer komponens alapú felépítést alkalmaz. Az alkalmazás különálló komponensekből épül fel, például bejelentkezési, regisztrációs és termékkezelő komponensekből. Ez a megközelítés növeli a kód áttekinthetőségét, újrafelhasználhatóságát és karbantarthatóságát.

3.5 CRUD műveletek

A rendszer támogatja a CRUD (Create, Read, Update, Delete) műveleteket. A felhasználó új termékeket hozhat létre, megtekintheti a meglévő termékeket, módosíthatja azok adatait, illetve törölheti őket. Ezek a funkciók biztosítják a terméknyilvántartó rendszer alapvető működését.

4. Felhasznált technológiák

A terméknyilvántartó rendszer fejlesztése során több korszerű webfejlesztési technológia került felhasználásra. A projekt kliens-szerver architektúrát alkalmaz, amelyben a frontend és backend egymástól elkülönülve működik.

4.1 Angular

Az alkalmazás kliensoldali részének fejlesztése Angular keretrendszer segítségével történt. Az Angular egy TypeScript alapú frontend keretrendszer, amely támogatja az egyoldalas webalkalmazások (SPA – Single Page Application) fejlesztését. Segítségével jól strukturált, komponens alapú alkalmazások hozhatók létre.

A projektben az Angular biztosítja:

- a felhasználói felület megjelenítését,
- az oldalak közötti navigációt,
- az űrlapok kezelését,
- valamint a backend szerverrel történő kommunikációt.

4.2 Angular Material

A felhasználói felület kialakításához Angular Material komponenskönyvtár került felhasználásra. Az Angular Material előre elkészített, modern megjelenésű komponenseket biztosít, amelyek gyorsabb és egységesebb felületfejlesztést tesznek lehetővé.

A projektben alkalmazott Angular Material elemek:

- gombok,
- beviteli mezők,
- kártyák,
- értesítő üzenetek (Snackbar),
- ikonok.

4.3 Node.js

A szerveroldali alkalmazás Node.js környezetben készült. A Node.js lehetővé teszi JavaScript futtatását szerveroldalon, így a frontend és backend fejlesztése ugyanazon programozási nyelv használatával történhet.

A backend feladata:

- a felhasználók hitelesítése,
- a termékek adatainak kezelése,
- a kliensoldali kérések feldolgozása,
- valamint az adatbázissal való kommunikáció.

4.4 Express.js

A backend fejlesztése során Express.js keretrendszer került alkalmazásra. Az Express.js egyszerű és hatékony megoldást biztosít REST API-k létrehozására.

A rendszerben az Express.js segítségével valósult meg:

- a bejelentkezési végpont,
- a regisztrációs végpont,
- a termékek lekérdezése,
- új termék hozzáadása,
- termék módosítása,
- termék törlése.

4.5 MongoDB

Az alkalmazás adatainak tárolása MongoDB adatbázisban történik. A MongoDB dokumentumorientált NoSQL adatbázis-kezelő rendszer, amely JSON-szerű dokumentumok formájában tárolja az adatokat.

A projektben két fő kollekció került kialakításra:

- Users (felhasználók),
- Products (termékek).

4.6 Mongoose

A MongoDB és a Node.js közötti kapcsolat kezelésére a Mongoose könyvtár került felhasználásra. A Mongoose lehetőséget biztosít sémák és modellek létrehozására, valamint egyszerűbb adatbázis műveletek végrehajtására.

Segítségével a termékek és felhasználók adatai strukturált módon kezelhetők és validálhatók.

5. A rendszer felépítése

A terméknyilvántartó rendszer kliens-szerver architektúrát alkalmaz. Az alkalmazás két fő részből áll: a frontendből és a backendből. A frontend felelős a felhasználói felület megjelenítéséért és a felhasználói interakciók kezeléséért, míg a backend biztosítja az üzleti logikát és az adatbázissal történő kommunikációt.

5.1 Frontend

A frontend Angular keretrendszer segítségével készült. Feladata a felhasználói felület megjelenítése, az adatok bekérése, valamint a backend szerverrel történő kommunikáció.

A frontend főbb elemei:

- Login oldal
- Regisztrációs oldal
- Termékkezelő felület
- Auth Guard útvonalvédelem
- Angular Material komponensek

A rendszerben a felhasználó először bejelentkezik vagy regisztrál, majd sikeres hitelesítés után elérheti a termékkezelő felületet.

5.2 Backend

A backend Node.js és Express.js segítségével készült. Feladata a kliensoldali kérések feldolgozása és az adatbázis kezelése.

A backend főbb funkciói:

- Felhasználók regisztrációja
- Bejelentkezés kezelése
- Termékek lekérdezése
- Új termék létrehozása
- Termék módosítása
- Termék törlése

A backend REST API végpontokon keresztül kommunikál a frontend alkalmazással.

5.3 Adatbázis

Az adatok MongoDB adatbázisban kerülnek tárolásra.

A rendszer két fő kollekciót használ:

5.3.1 Users

A felhasználók adatait tárolja.

Tárolt mezők:

- username
- password

5.3.2 Products

A termékek adatait tárolja.

Tárolt mezők:

- name
- category
- price
- stock

5.4 Projektstruktúra

5.4.1 Backend

backend/

```
├── config/
├── controllers/
├── models/
│   ├── Product.js
│   └── User.js
├── routes/
│   ├── authRoutes.js
│   └── productRoutes.js
├── server.js
└── package.json
```

A backend tartalmazza az adatbázis-modelleket, a REST API végpontokat, valamint a szerver konfigurációját. A Product és User modellek felelnek az adatok tárolásáért, míg a route fájlok kezelik a kliensoldalról érkező kéréseket.

5.4.2 Frontend

frontend/

```
├── src/
│   ├── app/
│   │   ├── guards/
│   │   ├── pages/
│   │   │   ├── login/
│   │   │   ├── register/
│   │   │   └── products/
│   │   ├── services/
│   │   ├── app.routes.ts
│   │   └── app.config.ts
│   ├── main.ts
│   ├── styles.css
│   └── material-theme.scss
└── package.json
```

A frontend Angular alapú alkalmazásként valósult meg. A pages mappában található az egyes oldalak (bejelentkezés, regisztráció, termékkezelés), a services mappa tartalmazza a backenddel kommunikáló szolgáltatásokat, míg a guards mappa az útvonalvédelem megvalósításáért felel.

6. Adatbázis tervezése

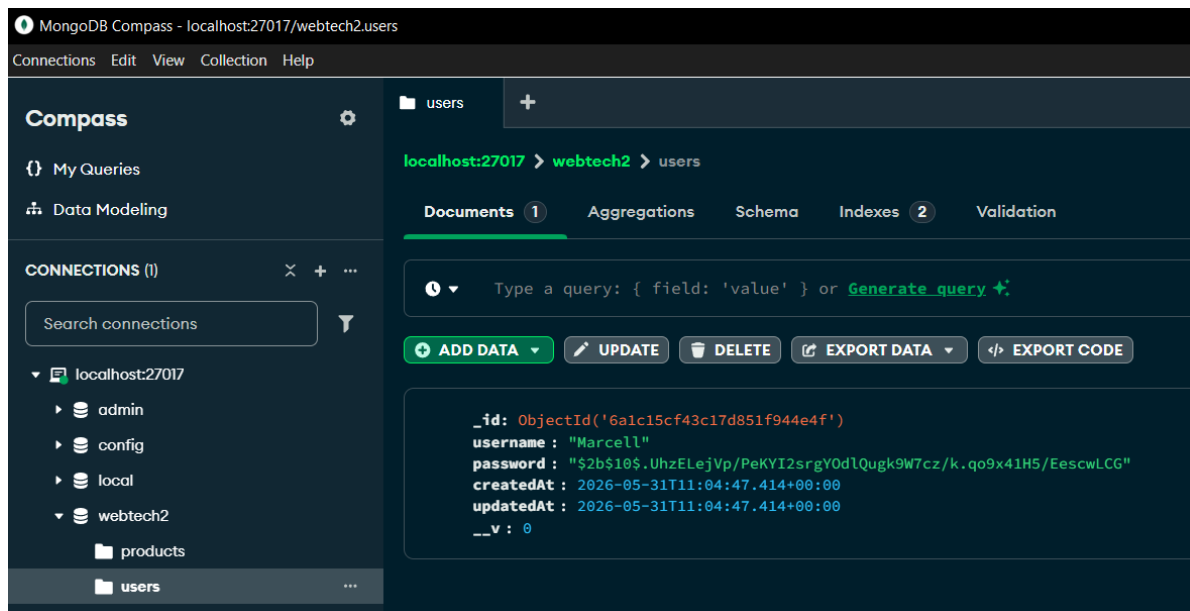
Az alkalmazás MongoDB adatbázist használ az adatok tárolására. Az adatbázis két fő kollekciónból áll: a felhasználók (Users) és a termékek (Products) kollekciónjából.

6.1 Users kollekción

A Users kollekción a rendszer felhasználóinak adatait tárolja. A felhasználók regisztráción során kerülnek rögzítésre az adatbázisban.

A kollekción mezőin:

Mező neve	Típus	Leírás
username	String	A felhasználó neve
password	String	A felhasználó jelszava



1. ábra – Users kollekción MongoDB Compassban

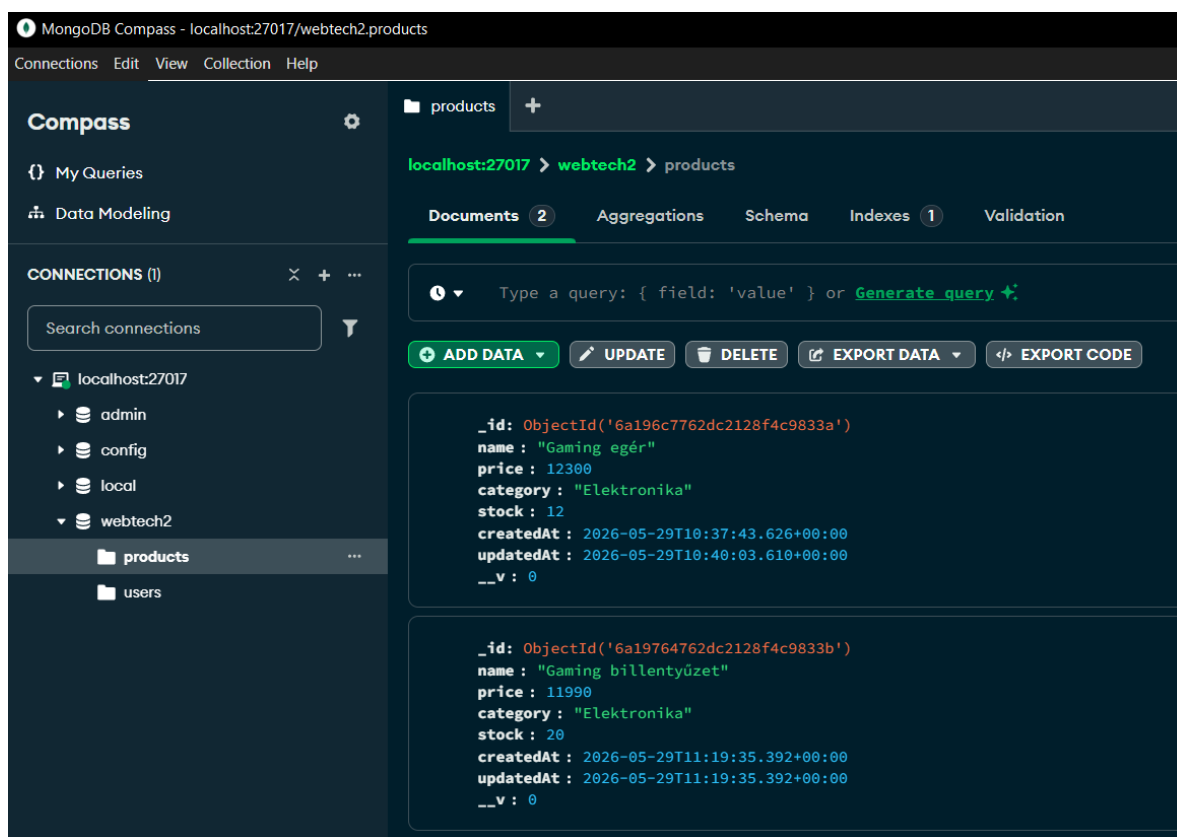
A MongoDB Compass felületén megtekinthető a Users kollekción, amely a rendszer felhasználóinak adatait tárolja. A képen látható, hogy a felhasználónév mellett a jelszó titkosított (hash-elt) formában kerül eltárolásra, ami növeli a rendszer biztonságát.

6.2 Products kollekció

A Products kollekció a nyilvántartott termékek adatait tartalmazza.

A kollekció mezői:

Mező neve	Típus	Leírás
name	String	A termék neve
category	String	A termék kategóriája
price	Number	A termék ára
stock	Number	A rendelkezésre álló készlet



2. ábra – Products kollekció MongoDB Compassban

A Products kollekció tartalmazza a rendszerben nyilvántartott termékek adatait. Minden dokumentum tárolja a termék nevét, kategóriáját, árát és készletinformációit. A MongoDB dokumentumorientált felépítése lehetővé teszi az adatok rugalmas kezelését.

6.3 Adatkapcsolatok

A jelenlegi rendszerben a Users és Products kollekciók között nincs közvetlen kapcsolat. A felhasználók a hitelesítéshez szükséges adatokat tárolják, míg a termékek a nyilvántartási funkciók megvalósítását szolgálják.

Az adatbázis egyszerű felépítése megfelel a projekt követelményeinek, ugyanakkor lehetőséget biztosít a rendszer későbbi bővítésére.

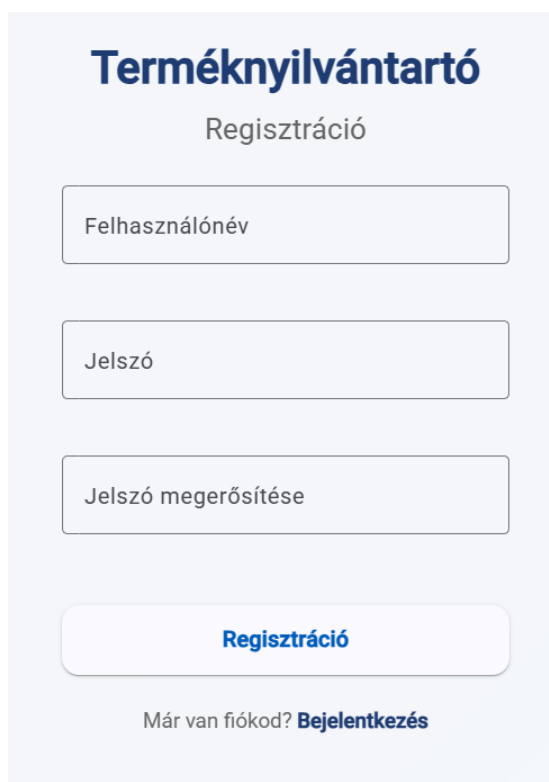
7. Megvalósított funkciók

A fejlesztés során a feladatkiírásban meghatározott funkciók mellett több kiegészítő szolgáltatás is megvalósításra került. A rendszer célja a termékek egyszerű és biztonságos nyilvántartása egy modern webes felületen keresztül.

7.1 Regisztráció

A rendszer lehetőséget biztosít új felhasználók regisztrációjára. A regisztráció során a felhasználó megadja a felhasználónevét és jelszavát, amelyeket a rendszer eltárol az adatbázisban.

A jelszavak nem kerülnek közvetlenül eltárolásra, hanem titkosított formában (hash) kerülnek mentésre, ezáltal növelve az alkalmazás biztonságát.



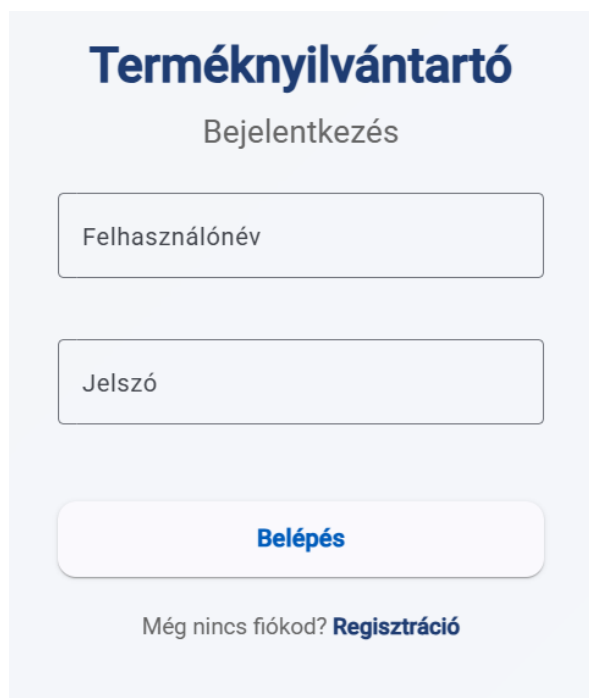
The image shows a registration form titled "Terméknyilvántartó" with the subtitle "Regisztráció". The form contains three input fields: "Felhasználónév", "Jelszó", and "Jelszó megerősítése". Below these fields is a blue "Regisztráció" button. At the bottom, there is a link that says "Már van fiókod? [Bejelentkezés](#)".

3. ábra - Regisztrációs felület

7.2 Bejelentkezés

A regisztrált felhasználók a bejelentkezési oldalon tudnak belépni a rendszerbe. A rendszer ellenőrzi a megadott adatokat, és sikeres hitelesítés esetén hozzáférést biztosít a termékkezelő felülethez.

Sikertelen bejelentkezés esetén hibaüzenet jelenik meg.



The screenshot shows a login interface for a system titled "Terméknyilvántartó". Below the title is the subtitle "Bejelentkezés". There are two input fields: the first is labeled "Felhasználónév" (Username) and the second is labeled "Jelszó" (Password). Below these fields is a blue button labeled "Belépés" (Login). At the bottom, there is a link that says "Még nincs fiókod? Regisztráció" (Don't have an account? Registration).

4. ábra – Bejelentkezési felület

7.3 Navigációs fejléc és felhasználói információk

A termékkezelő felület tetején egy navigációs fejléc található, amely megjeleníti az alkalmazás nevét, a bejelentkezett felhasználó nevét, valamint a kijelentkezés lehetőségét.

A felhasználónév dinamikusan kerül megjelenítésre a bejelentkezés során eltárolt adatok alapján. Ez egyértelmű visszajelzést ad a felhasználónak arról, hogy sikeresen bejelentkezett a rendszerbe.

A fejléc jobb oldalán található Kijelentkezés gomb lehetővé teszi a munkamenet biztonságos lezárását. A kijelentkezés során a rendszer törli a bejelentkezéshez szükséges adatokat, majd visszairányítja a felhasználót a bejelentkezési oldalra.



5. ábra – Navigációs fejléc és felhasználói információk

7.4 Útvonalvédelem (Auth Guard)

A rendszer Auth Guard segítségével védi a termékkezelő oldalt. Amennyiben a felhasználó nincs bejelentkezve, a rendszer automatikusan visszairányítja a bejelentkezési oldalra.

Ez megakadályozza, hogy illetéktelen felhasználók hozzáférjenek a nyilvántartott adatokhoz.

7.5 Termékek megjelenítése

A termékek listája táblázatos formában jelenik meg. A felhasználó áttekintheti a rendszerben tárolt összes terméket, azok kategóriáját, árát és készletinformációit.

Termékek listája

Kategória szűrése

Összes kategória

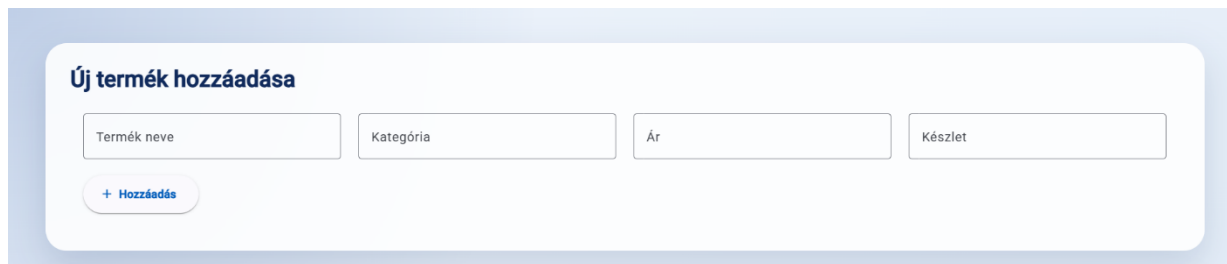
Név	Kategória	Ár	Készlet	Műveletek
Kapucnis pulóver	Ruházat	12990 Ft	10 db	<a>Szerkesztés <a>Törlés
Golyóstoll	Irodaszer	450 Ft	120 db	<a>Szerkesztés <a>Törlés
Energiaital	Élelmiszer	699 Ft	60 db	<a>Szerkesztés <a>Törlés
LEGO Speed Champions	Játék	12990 Ft	15 db	<a>Szerkesztés <a>Törlés
Csokoládé	Élelmiszer	890 Ft	35 db	<a>Szerkesztés <a>Törlés
A4 fénymásolópapír	Irodaszer	1990 Ft	50 db	<a>Szerkesztés <a>Törlés
Pamut póló (Fekete, M)	Ruházat	4000 Ft	120 db	<a>Szerkesztés <a>Törlés
Vezeték nélküli egér X-100	Elektronika	8000 Ft	45 db	<a>Szerkesztés <a>Törlés
Gaming billentyűzet	Elektronika	28900 Ft	8 db	<a>Szerkesztés <a>Törlés
Gaming egér	Elektronika	12300 Ft	12 db	<a>Szerkesztés <a>Törlés

6. ábra - Terméklista

7.6 Új termék hozzáadása

A felhasználó új termékeket vehet fel a rendszerbe egy űrlap segítségével. A rendszer ellenőrzi a megadott adatokat, és csak érvényes adatok esetén engedélyezi a mentést.

Sikeres mentés után a termék azonnal megjelenik a listában.

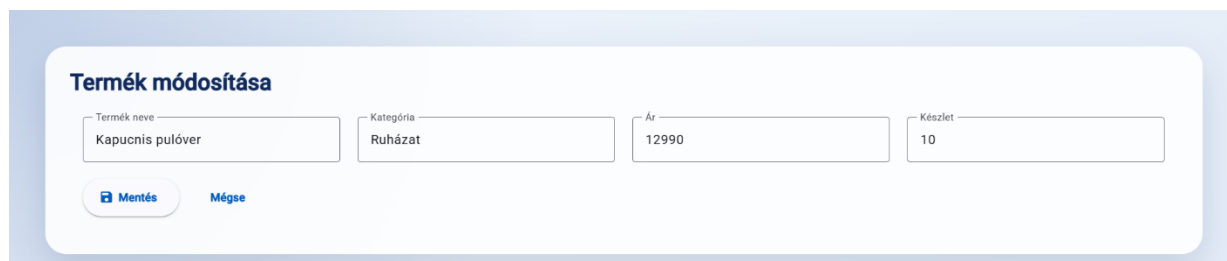
A screenshot of the 'Új termék hozzáadása' (Add new product) form. It features a light blue header with the title. Below the title are four input fields: 'Termék neve' (Product name), 'Kategória' (Category), 'Ár' (Price), and 'Készlet' (Stock). A blue button with a plus icon and the text '+ Hozzáadás' (Add) is positioned below the first field.

7. ábra - Termék hozzáadása felület

7.7 Termék módosítása

A már meglévő termékek adatai szerkeszthetők. A módosítás során a felhasználó átírhatja a termék nevét, kategóriáját, árát vagy készletét.

A változtatások mentése után az adatok frissülnek az adatbázisban.

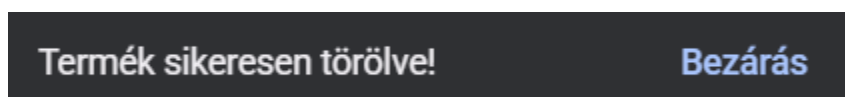
A screenshot of the 'Termék módosítása' (Edit product) form. It has a light blue header with the title. Below the title are four input fields with pre-filled data: 'Termék neve' (Kapucnis pulóver), 'Kategória' (Ruházat), 'Ár' (12990), and 'Készlet' (10). At the bottom left, there are two buttons: a blue 'Mentés' (Save) button with a save icon and a grey 'Mégse' (Cancel) button.

8. ábra - Termék szerkesztése felület

7.8 Termék törlése

A rendszer lehetőséget biztosít a termékek törlésére. A törlés során a kiválasztott termék eltávolításra kerül az adatbázisból és a felületről is.

A sikeres műveletről a rendszer visszajelzést ad a felhasználó számára.

A screenshot of a dark grey notification banner. On the left, it says 'Termék sikeresen törölve!' (Product successfully deleted!) in white text. On the right, there is a blue button with the text 'Bezárás' (Close).

9. ábra - Termék törlésének visszajelzése

7.9 Snackbar értesítések

A felhasználói élmény javítása érdekében Angular Material Snackbar értesítések kerültek alkalmazásra.

Az értesítések tájékoztatják a felhasználót:

- sikeres hozzáadásról,
- sikeres módosításról,
- sikeres törlésről,
- esetleges hibákról.

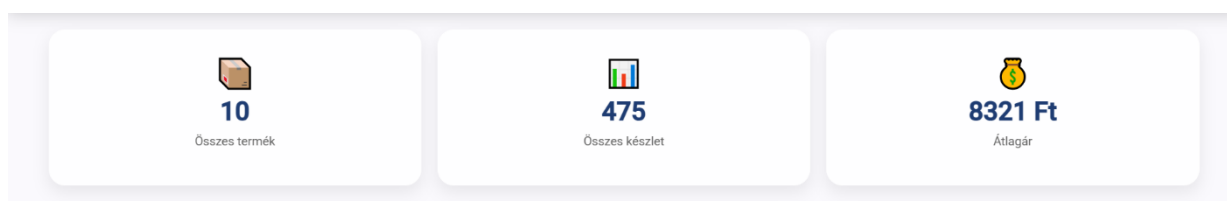
7.10 Statisztikai kártyák

A termékkezelő felület tetején statisztikai kártyák jelennek meg, amelyek összefoglalják a nyilvántartás legfontosabb adatait.

A kártyák megjelenítik:

- az összes termék számát,
- az összes készlet mennyiségét,
- az átlagos termékárát.

A kártyák valós időben frissülnek a termékadatok változásakor.



10. ábra – Statisztikai kártyák

7.11 Kategória szerinti szűrés

A rendszer lehetőséget biztosít a termékek kategória szerinti szűrésére. A felhasználó egy lenyíló lista segítségével kiválaszthatja a kívánt kategóriát, amely után a táblázat csak az adott kategóriába tartozó termékeket jeleníti meg.

A szűrés dinamikusan működik, így a terméklista azonnal frissül a kiválasztott kategóriának megfelelően. A statisztikai kártyák szintén a szűrt adatok alapján számolják az összesített értékeket.

A kategóriák megjelenítése színezett jelölőkkel (badge-ekkel) történik, amelyek vizuálisan is segítik a különböző kategóriák elkülönítését.



The screenshot shows a web interface titled 'Termékek listája'. In the top right corner, there is a dropdown menu labeled 'Kategória szűrése' with 'Ruházat' selected. Below this is a table with the following columns: 'Név', 'Kategória', 'Ár', 'Készlet', and 'Műveletek'. The table contains two rows of products, both of which have a pink 'Ruházat' badge in the 'Kategória' column.

Név	Kategória	Ár	Készlet	Műveletek
Kapucnis pulóver	Ruházat	12990 Ft	10 db	Szerkesztés Törölés
Pamut póló (Fekete, M)	Ruházat	4000 Ft	120 db	Szerkesztés Törölés

11. ábra – Ruházat szerinti szűrés



The screenshot shows the same web interface as before, but the dropdown menu 'Kategória szűrése' now has 'Irodaszer' selected. The table below it now displays two rows of products, both of which have an orange 'Irodaszer' badge in the 'Kategória' column.

Név	Kategória	Ár	Készlet	Műveletek
Golyóstoll	Irodaszer	450 Ft	120 db	Szerkesztés Törölés
A4 fénymásolópapír	Irodaszer	1990 Ft	50 db	Szerkesztés Törölés

12. ábra – Irodaszer szerinti szűrés

8. Fontosabb kódrészletek bemutatása

8.1 Felhasználó regisztrációja

```
router.post('/register', async (req, res) => {
  try {
    const { username, password } = req.body;

    if (!username || !password) {
      return res.status(400).json({ message: 'Felhasználónév és jelszó megadása kötelező' });
    }

    const existingUser = await User.findOne({ username });

    if (existingUser) {
      return res.status(400).json({ message: 'Ez a felhasználónév már létezik' });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    const user = new User({
      username,
      password: hashedPassword
    });

    await user.save();

    res.status(201).json({ message: 'Sikeres regisztráció' });
  } catch (error) {
    res.status(500).json({ message: 'Hiba a regisztráció során' });
  }
});
```

Magyarázat: A regisztráció során a rendszer ellenőrzi, hogy a megadott felhasználónév már létezik-e. Amennyiben nem, a jelszó bcrypt segítségével titkosításra kerül, majd az új felhasználó elmentésre kerül a MongoDB adatbázisba.

8.2 Felhasználó bejelentkezése

```
router.post('/login', async (req, res) => {
  try {
    const { username, password } = req.body;

    if (!username || !password) {
      return res.status(400).json({ message: 'Felhasználónév és jelszó megadása kötelező' });
    }

    const user = await User.findOne({ username });

    if (!user) {
      return res.status(401).json({ message: 'Hibás felhasználónév vagy jelszó' });
    }

    const isPasswordCorrect = await bcrypt.compare(password, user.password);

    if (!isPasswordCorrect) {
      return res.status(401).json({ message: 'Hibás felhasználónév vagy jelszó' });
    }

    res.json({
      message: 'Sikeres bejelentkezés',
      username: user.username
    });
  } catch (error) {
    res.status(500).json({ message: 'Hiba a bejelentkezés során' });
  }
});

module.exports = router;
```

Magyarázat: A bejelentkezés során a rendszer megkeresi a felhasználót az adatbázisban, majd összehasonlítja a megadott jelszót a titkosított jelszóval. Sikeres hitelesítés esetén a felhasználó hozzáférést kap a termékkezelő felülethez.

8.3 Termék betöltése

```
loadProducts(): void {
  this.productService.getProducts().subscribe({
    next: (data) => {
      this.products = data;
    }
  });
}
```

Magyarázat: A `loadProducts()` metódus a `ProductService` segítségével lekéri a backendtől az adatbázisban tárolt termékeket. A válaszként kapott adatok a **products** tömbbe kerülnek, amelyet a táblázat használ megjelenítésre.

8.4 Termék hozzáadása / módosítása

```
saveProduct(): void {
  if (!this.product.name || !this.product.category || this.product.price <= 0 || this.product.stock < 0) {
    this.showMessage('Kérlek, tölts ki minden mezőt helyesen!');
    return;
  }

  if (this.editingId) {
    this.productService.updateProduct(this.editingId, this.product).subscribe({
      next: () => {
        this.loadProducts();
        this.resetForm();
        this.showMessage('Termék sikeresen módosítva!');
      }
    });
  } else {
    this.productService.addProduct(this.product).subscribe({
      next: () => {
        this.loadProducts();
        this.resetForm();
        this.showMessage('Termék sikeresen hozzáadva!');
      }
    });
  }
}
```

Magyarázat: A `saveProduct()` metódus kezeli az új termék hozzáadását és a meglévő termék módosítását is. Először ellenőrzi, hogy a kötelező mezők megfelelően ki vannak-e töltve. Ha az `editingId` értéke nem üres, akkor módosítás történik, ellenkező esetben új termék kerül rögzítésre.

8.5 Kategória szerinti szűrés

```
get filteredProducts(): Product[] {
  if (!this.selectedCategory) {
    return this.products;
  }

  return this.products.filter(
    p => p.category === this.selectedCategory
  );
}
```

Magyarázat: A `filteredProducts()` getter biztosítja a kategória szerinti szűrés működését. Ha nincs kiválasztott kategória, akkor az összes termék megjelenik. Ellenkező esetben csak azok a termékek kerülnek visszaadásra, amelyek kategóriája megegyezik a kiválasztott értékkel.

8.6 Kategóriák színeztése

```
getCategoryClass(category: string): string {  
  switch (category.toLowerCase()) {  
    case 'elektronika':  
      return 'electronics-chip';  
  
    case 'játék':  
      return 'games-chip';  
  
    case 'ruházat':  
      return 'clothing-chip';  
  
    case 'irodaszer':  
      return 'office-chip';  
  
    case 'élelmiszer':  
      return 'food-chip';  
  
    default:  
      return 'default-chip';  
  }  
}
```

Magyarázat: A getCategoryClass() metódus a termék kategóriája alapján CSS osztályt rendel az adott kategóriajelölőhöz. Ennek segítségével a különböző kategóriák eltérő színnel jelennek meg a táblázatban.

8.7 Statisztikai kártyák számítása

```
get totalProducts(): number {
  return this.filteredProducts.length;
}

get totalStock(): number {
  return this.filteredProducts.reduce((sum, p) => sum + p.stock, 0);
}

get averagePrice(): number {
  if (this.filteredProducts.length === 0) return 0;

  const total = this.filteredProducts.reduce(
    (sum, p) => sum + p.price,
    0
  );

  return Math.round(total / this.filteredProducts.length);
}
```

Magyarázat: A statisztikai kártyák getterek segítségével számolják ki az összes termék számát, az összes készlet mennyiségét és az átlagárát. A számítások a szűrt terméklistán alapulnak, ezért kategóriaszűrés esetén a kártyák értékei is automatikusan frissülnek.

8.8 Kijelentkezés

```
logout(): void {
  this.authService.logout();
  this.router.navigate(['/']);
}
```

Magyarázat: A `logout()` metódus kijelentkezteti a felhasználót, majd visszairányítja a bejelentkezési oldalra.